

LINUX ASSIGNMENT

Aditi Biju

261100690027

Hard Link vs. Soft Link

Part A — Create the links

Create a directory named `link_lab` and navigate to it.

Create a file named `security.txt` containing the following text:

Linux File System Security Lab

Confidential Security Report

Create:

a hard link named `security_hard.txt`

a symbolic link named `security_soft.txt`

List all three files using a command that displays their inode numbers.

Record the inode numbers of:

`security.txt`

`security_hard.txt`

`security_soft.txt`

```
(aduuu@kali)~$ mkdir link_lab
(aduuu@kali)~$ cd link_lab
(aduuu@kali)~/link_lab$ echo -e "Linux File System Security Lab\nConfidential Security Report" > security.txt
(aduuu@kali)~/link_lab$ ln security.txt security_hard.txt
(aduuu@kali)~/link_lab$ ln -s security.txt security_soft.txt
(aduuu@kali)~/link_lab$ ls -li
total 8
1372259 -rw-rw-r-- 2 aduuu aduuu 60 Sep  4 00:39 security_hard.txt
1372260 lrwxrwxrwx 1 aduuu aduuu 12 Sep  4 00:39 security_soft.txt -> security.txt
1372259 -rw-rw-r-- 2 aduuu aduuu 60 Sep  4 00:39 security.txt
(aduuu@kali)~/link_lab$ cat security.txt
Linux File System Security Lab
Confidential Security Report
```

What difference do you observe?

security.txt and security_hard.txt have the same inode number because they are two names for the same file.

security_soft.txt has a different inode number because it is a separate file that simply stores the path to security.txt.

Part B — Examine File Metadata

Use `ls -l` to examine the three files. Identify the file type of each file from the first character of the permission field.

Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

```
(aduuu@kali)~[Link_lab]
$ ls -l
total 8
-rw-rw-r-- 2 aduuu aduuu 60 Sep  4 00:39 security_hard.txt
lrwxrwxrwx 1 aduuu aduuu 12 Sep  4 00:39 security_soft.txt -> security.txt
-rw-rw-r-- 2 aduuu aduuu 60 Sep  4 00:39 security.txt

(aduuu@kali)~[Link_lab]
$ stat security.txt
  File: security.txt
  Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1        Inode: 1372259    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  aduuu)   Gid: ( 1000/  aduuu)
Access: 2026-09-04 00:40:45.821284773 +0530
Modify: 2026-09-04 00:39:13.074353118 +0530
Change: 2026-09-04 00:39:20.545164170 +0530
 Birth: 2026-09-04 00:39:13.074353118 +0530

(aduuu@kali)~[Link_lab]
$ stat security_hard.txt
  File: security_hard.txt
  Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1        Inode: 1372259    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  aduuu)   Gid: ( 1000/  aduuu)
Access: 2026-09-04 00:40:45.821284773 +0530
Modify: 2026-09-04 00:39:13.074353118 +0530
Change: 2026-09-04 00:39:20.545164170 +0530
 Birth: 2026-09-04 00:39:13.074353118 +0530

(aduuu@kali)~[Link_lab]
$ stat security_soft.txt
  File: security_soft.txt -> security.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,1        Inode: 1372260    Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/  aduuu)   Gid: ( 1000/  aduuu)
Access: 2026-09-04 00:39:32.343400626 +0530
Modify: 2026-09-04 00:39:27.560161030 +0530
Change: 2026-09-04 00:39:27.560161030 +0530
 Birth: 2026-09-04 00:39:27.560161030 +0530
```

Use the `stat` command on all three files and record the following information:

Parameter	Original	Hard Link	Soft Link
Inode number	1372259	1372259	1372260
File type	Regular file	Regular file	Symbolic link
Size	60 bytes	60 bytes	12 bytes
Link count	2	2	1
Permissions	0664	0664	0664

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

The original file and the hard link are both 60 bytes because they point to the same file. The soft link is only 12 bytes because it stores the name/path of the original file, not its actual contents.

File type — first character of permission field

File	First char	Type
security.txt	-	Regular file
security_hard.txt	-	Regular file
security_soft.txt	1	Symbolic link

Part C — Investigate readlink

1. Execute readlink on security_soft.txt. What information is displayed?

```
readlink security_soft.txt
```

The output is: security.txt

This shows that security_soft.txt is a symbolic link pointing to security.txt.

2. Execute readlink on security_hard.txt. What do you observe?

```
readlink security_hard.txt
```

Observation:

There is no output because security_hard.txt is a hard link. A hard link does not store the pathname of the original file; it directly refers to the same inode.

3. Based on your observations, explain why readlink works differently for hard links and symbolic links.

Use: readlink -f security_soft.txt

It displays the complete absolute path of the final target of the symbolic link, for example:

```
/home/aduuu/link_lab/security.txt
```

Explanation:

readlink works differently because a symbolic link stores the pathname of its target, so readlink can display that pathname. A hard link does not store a target pathname; it directly refers to the same inode as the original file, so readlink security_hard.txt produces no output. readlink -f follows the symbolic link and resolves it to the final absolute path.

4. What does the -f option reveal?

The -f option reveals the complete absolute path of the final target of the symbolic link.

```
(aduuu@kali)-[~/link_lab]
$ readlink security_soft.txt
security.txt

(aduuu@kali)-[~/link_lab]
$ readlink security_hard.txt

(aduuu@kali)-[~/link_lab]
$ readlink -f security_soft.txt
/home/aduuu/link_lab/security.txt
```

Part D — Content Modification

1. Append the following line to security.txt: Unauthorized login detected

This appends the line to security.txt without deleting the existing content.

2. Display the contents using all three filenames.

```
cat security.txt
```

```
cat security_hard.txt
```

```
cat security_soft.txt
```

The line Unauthorized login detected appears in all three outputs, because the hard link and symbolic link both access the same file content.

3. Is the newly added content visible through both links? Explain why

Yes. The newly added content is visible through both links.

- security_hard.txt is a hard link to the same inode and file data as security.txt.
- security_soft.txt is a symbolic link that points to security.txt.

Therefore, when content is added to security.txt, the updated content is accessible through both links.

```
(aduuu@kali)~/link_lab
$ readlink security_soft.txt
security.txt

(aduuu@kali)~/link_lab
$ readlink security_hard.txt

(aduuu@kali)~/link_lab
$ readlink -f security_soft.txt
/home/aduuu/link_lab/security.txt

(aduuu@kali)~/link_lab
$ echo "Unauthorized login detected" >> security.txt
```

```
(aduuu@kali)~/link_lab
$ readlink security_soft.txt
security.txt

(aduuu@kali)~/link_lab
$ readlink security_hard.txt

(aduuu@kali)~/link_lab
$ readlink -f security_soft.txt
/home/aduuu/link_lab/security.txt

(aduuu@kali)~/link_lab
$ echo "Unauthorized login detected" >> security.txt

(aduuu@kali)~/link_lab
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(aduuu@kali)~/link_lab
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(aduuu@kali)~/link_lab
$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(aduuu@kali)~/link_lab
$ echo "System security alert" >> security_hard.txt

(aduuu@kali)~/link_lab
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
```

```
(aduuu@kali)~
$ cd link_lab

(aduuu@kali)~/link_lab
$ ls
incident.txt  security_hard.txt  security_soft.txt
```

Part E — Rename Experiment

security.txt

to:

incident.txt

1. Try accessing the content through security_hard.txt. Does it still work?

```
mv security.txt incident.txt
```

2. Try accessing the content through security_soft.txt. Does it still work?

```
cat security_hard.txt
```

Yes. The content is still accessible through the hard link.

3. Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?

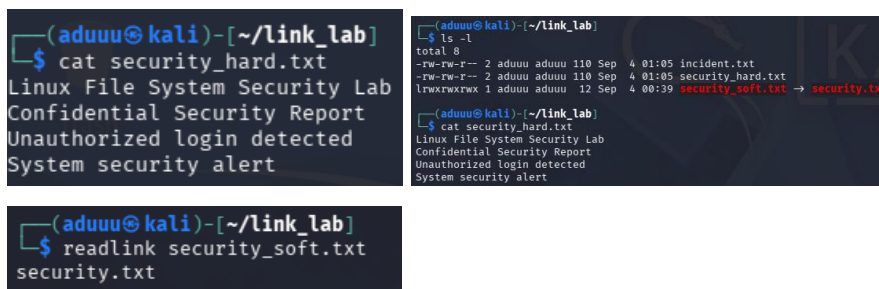
It will display: security.txt

This is because the symbolic link stores the original pathname, even though the file was renamed to incident.txt.

4. Explain why renaming the original file affects the symbolic link differently from the hard link.

Hard link: It points directly to the same inode, so renaming the original filename does not affect the hard link. It continues to work.

Symbolic link: It stores the pathname of the target (security.txt). When the original file is renamed to incident.txt, that pathname no longer exists, so the symbolic link becomes broken.



The image contains three terminal screenshots. The top-left screenshot shows a user at a Kali machine in the directory ~/link_lab running 'cat security_hard.txt', which outputs a security alert message. The top-right screenshot shows the same user running 'ls -l', displaying a list of files including incident.txt, security_hard.txt, and a broken symbolic link security_soft.txt pointing to security.txt. The bottom screenshot shows the user running 'readlink security_soft.txt', which outputs 'security.txt'.

```
(aduuu@kali)-[~/link_lab]
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
System security alert

(aduuu@kali)-[~/link_lab]
$ ls -l
total 8
-rw-rw-r-- 2 aduuu aduuu 110 Sep  4 01:05 incident.txt
-rw-rw-r-- 2 aduuu aduuu 110 Sep  4 01:05 security_hard.txt
lrwxrwxrwx 1 aduuu aduuu 12 Sep  4 00:39 security_soft.txt -> security.txt

(aduuu@kali)-[~/link_lab]
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
System security alert

(aduuu@kali)-[~/link_lab]
$ readlink security_soft.txt
security.txt
```

Part F — Deletion Experiment

1. Recreate the links if necessary and then perform the following.

```
security_hard.txt
```

2. Delete the original file.

```
rm security.txt
```

3. Check whether security_hard.txt can still be accessed.

```
cat security_hard.txt
```

Yes, security_hard.txt can still be accessed and will display the file contents. This is because the hard link refers directly to the same inode, even after the original filename is deleted.

4. Check whether security_soft.txt can still be accessed.

```
cat security_soft.txt
```

No, security_soft.txt cannot be accessed. It will give a “No such file or directory” error because its target security.txt has been deleted. The symbolic link becomes a dangling/broken link.

5. Use ls -l and readlink to investigate the symbolic link after deleting its target.

```
ls -l security_soft.txt
```

```
readlink security_soft.txt
```

ls -l shows:

```
security_soft.txt -> security.txt
```

This shows the symbolic link still exists, but its target has been deleted.

readlink displays: security.txt

This confirms that the link still stores the old target path.

security_soft.txt is now a dangling/broken symbolic link.

6. What is a dangling/broken symbolic link?

A dangling/broken symbolic link is a symbolic link whose target file or path no longer exists.

7. Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.

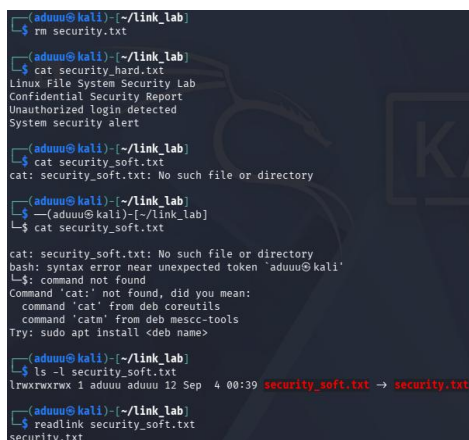
No. Deleting the original filename does not immediately delete the underlying data when a hard link exists.

Before deletion, the inode has a link count of 2 (security.txt and security_hard.txt).

After deleting security.txt, the link count becomes 1.

The same inode and underlying data remain, so security_hard.txt can still access the contents.

The data is removed only when the link count reaches 0 (and there are no open references).



```
(aduuu@kali)~/link_lab
└─$ rm security.txt

(aduuu@kali)~/link_lab
└─$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
System security alert

(aduuu@kali)~/link_lab
└─$ cat security_soft.txt
cat: security_soft.txt: No such file or directory

(aduuu@kali)~/link_lab
└─$ ls -l security_soft.txt
lrwxrwxrwx 1 aduuu aduuu 12 Sep  4 00:39 security_soft.txt -> security.txt

(aduuu@kali)~/link_lab
└─$ readlink security_soft.txt
security.txt
```

Part G — Directory and Filesystem Behaviour

1. Create a directory named evidence. Attempt to create a hard link to this directory. Record the result.

`mkdir evidence`

`ln evidence evidence_hard`

Expected result:

ln: evidence: hard link not allowed for directory

The hard link creation fails because hard links to directories are not normally allowed.

2. Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?

Yes, the operation is successful.

`ln -s evidence evidence_link`

The symbolic link evidence_link is successfully created and points to the evidence directory. Symbolic links can point to directories.

3. Based on your experiment, compare hard and symbolic links with respect to directory linking.

Hard link: Cannot normally be created for a directory. The attempt fails.

Symbolic link: Can be created for a directory successfully. evidence_link points to the evidence directory.

If multiple filesystems/mount points are available in the laboratory environment, attempt to create:

a hard link across filesystems

A hard link cannot be created across different filesystems. The operation should fail with an error such as: Invalid cross-device link

a symbolic link across filesystems

A symbolic link can be created across different filesystems: `ln -s /path/on/other/filesystem evidence_link`

Hard links must be on the same filesystem because they refer to the same inode. Symbolic links can cross filesystems because they store the target pathname.

Final Comparison

Parameter	Hard Link	Symbolic Link
Inode number	Same as target	Different from target
File type	Regular file	Symbolic link
Link count	Increases	Usually 1
File size	Same as target	Size of stored pathname
Stores target pathname	No	Yes
readlink output	No output	Shows target pathname
Effect of modifying target	Change is visible	Change is visible
Effect of renaming target	Still works	Becomes broken
Effect of deleting target	Still works if another hard link exists	Becomes broken
Can become a dangling or broken link?	No	Yes
Directory linking	Not normally allowed	Allowed
Cross-filesystem linking	Not allowed	Allowed

sort

Suppose alerts.txt contains:

192.168.1.10 admin 5

10.0.0.8 root 12

172.16.0.5 guest 2

10.0.0.20 admin 8

Format:

IP-address username failed attempts

Sort according to failed attempts

```
(aduuu@kali)-[~]
$ nano alerts.txt

(aduuu@kali)-[~]
$ sort -k3 -n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

Sort a list of suspicious IP addresses alphabetically.

```
(aduuu@kali)-[~]
$ nano suspicious_ips.txt

(aduuu@kali)-[~]
$ sort suspicious_ips.txt

10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10

(aduuu@kali)-[~]
$ sort -u suspicious_ips.txt

10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```


Display only unique suspicious IP addresses using sort.

```
(aduuu@kali)-[~]  
$ sort -u suspicious_ips.txt  
  
10.0.0.20  
10.0.0.8  
172.16.0.5  
192.168.1.10
```

sort -u sorts the suspicious IP addresses and displays only unique IP addresses.

Sort a list of detected port numbers numerically.

```
(aduuu@kali)-[~]  
$ nano ports.txt  
  
(aduuu@kali)-[~]  
$ sort -n ports.txt  
  
21  
22  
80  
443  
8080  
  
(aduuu@kali)-[~]  
$ sort -nr ports.txt  
  
8080  
443  
80  
22  
21
```

sort -n sorts the detected port numbers in numerical order from smallest to largest.

Display port numbers in descending numerical order.

```
(aduuu@kali)-[~]  
$ sort -nr ports.txt  
  
8080  
443  
80  
22  
21
```

Sort usernames without considering uppercase/lowercase differences.

```
(aduuu@kali)-[~]  
$ sort -f usernames.txt  
  
alice  
Alice  
bob  
BOB  
charlie  
CHARLIE
```

From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

```
(aduuu@kali)-[~]  
$ nano failed_login.log  
  
(aduuu@kali)-[~]  
$ sort failed_login.log | uniq -c | sort -nr  
  
3 192.168.1.10  
2 10.0.0.8  
1 172.16.0.5
```

Given IP,username,failed_attempts, sort records based on the number of failed attempts.

```
(aduuu@kali)~  
$ nano records.csv  
  
(aduuu@kali)~  
$ sort -t',' -k3 -n records.csv  
172.16.0.5,guest,2  
192.168.1.10,admin,5  
10.0.0.20,admin,8  
10.0.0.8,root,12
```

The records are sorted in ascending order based on the third field, which represents the number of failed attempts.

Sort a CSV vulnerability report based on severity score.

```
(aduuu@kali)~  
$ nano vulnerabilities.csv  
  
(aduuu@kali)~  
$ sort -t',' -k3 -n vulnerabilities.csv  
CVE-004,Path Traversal,4  
CVE-002,XSS,6  
CVE-001,SQL Injection,9  
CVE-003,Command Injection,10
```

Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.

```
(aduuu@kali)~  
$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' failed_login.log | sort -u  
10.0.0.8  
172.16.0.5  
192.168.1.10
```

The command extracts IP addresses from the log, sorts them, and displays only unique IP addresses.

Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
(aduuu@kali)~  
$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' failed_login.log | sort | uniq -c | sort -nr | head -5  
3 192.168.1.10  
2 10.0.0.8  
1 172.16.0.5
```

The pipeline extracts IP addresses, counts their occurrences, sorts them by frequency in descending order, and displays the top 5.